Predicting One-Hour Drift for a Station-Keeping USV

Jake Cushway
University of Victoria
Victoria, Canada
jakecush1@uvic.ca

Rowan Hall
University of Victoria
Victoria, Canada
rowanhall@uvic.ca

Abstract

Between course corrections, the DataXplorer™ USV drifts with its motor off, and a pilot must step in before wind and current carry it out of its 10 nm station. We predict where it will be in one hour, using only its last hour of GPS track and wind. Linear fits show wind speed explains under 15% of drift, and past drift most of it. We compare four models (persistence, linear regression, LSTM, transformer) on an unseen deployment. Code: https://github.com/jakecush1/autonomous-pilot-ML (`usv-intervention/solution`).[1]

Keywords

USV, drift prediction, time series regression, LSTM, transformer

1 Introduction

The Open Ocean Robotics DataXplorer replaces a retired NOAA weather buoy and must stay within 10 nm of the buoy's position. It mostly drifts with the motor off, and a pilot watches it 24/7 although a correction is needed only every 2–3 hours. Milestones 1 and 2 set out to predict *when* the pilot must act. The first step is predicting *where* the drifting vessel is going: a position predicted an hour ahead can be checked against the 10 nm limit to alert the pilot early. Unlike drift models built on physics and weather forecasts [1, 5], we learn drift from the vessel's own telemetry. We contribute a formal task definition, an analysis of what drives drift, a leak-free data pipeline, and a comparison of four models.

2 Problem Definition

A deployment is a sequence of 1-minute telemetry records $\mathcal{S} = \{s_\tau\}_{\tau=1}^{T}$. As in Milestone 1, $s_\tau = (p_\tau, v_\tau, \theta_\tau, m_\tau)$ holds position (latitude, longitude), wind speed (kn) and the direction the wind blows *from*, plus motor power (W). The vessel is *drifting* at minute τ if $m_\tau = 0$. Let $\mathbf{p}_\tau \in \mathbb{R}^2$ be its position in metres east and north of the station p_0 (§3.1). For a time t at which the vessel has drifted for the past H minutes, our solution is a function

$$g : \mathbb{R}^{H\times 4} \to \mathbb{R}^2, \qquad \hat{\mathbf{y}}_t = g(\mathbf{X}_t) \approx \mathbf{y}_t = \mathbf{p}_{t+H} - \mathbf{p}_t, \tag{1}$$

where $\mathbf{X}_t$ describes the past H minutes (4 numbers per minute) and $\mathbf{y}_t$ is the movement over the next H minutes, assuming the motor stays off. The predicted position $\hat{\mathbf{p}}_{t+H} = \mathbf{p}_t + \hat{\mathbf{y}}_t$ can then feed the decision f of Milestone 1, which checks it against the limit $R = 10$ nm. The output is a distance, not a category, so this is regression, not classification. We use $H = 60$ (§3.2).

3 Proposed Approach

Figure 1 shows the pipeline and Algorithm 1 gives the pseudo-code.

[1]AI use (Guideline D): the code and a first draft of this paper were generated with Claude (Anthropic) and reviewed, run and edited by the authors.

3.1 Representation

Position. Positions $(\phi_\tau, \lambda_\tau)$ are converted to metres from the station $p_0 = (\phi_0, \lambda_0)$, so that distances and movements are meaningful: $\mathbf{p}_\tau = r\,\big((\lambda_\tau - \lambda_0)\cos\phi_0,\ \phi_\tau - \phi_0\big)$, with Earth radius $r = 6371$ km and angles in radians. This flat-earth approximation is off by less than 0.3% within 10 nm. **Wind.** The wind blowing *from* direction θ pushes the vessel the opposite way, so we use the push vector $\mathbf{w}_\tau = -v_\tau(\sin\theta_\tau, \cos\theta_\tau)$ (east, north). **Input window.** For each minute we keep its movement $\Delta\mathbf{p}_\tau = \mathbf{p}_\tau - \mathbf{p}_{\tau-1}$ and the wind push, giving $\mathbf{X}_t = [\ (\Delta\mathbf{p}_\tau, \mathbf{w}_\tau)\ ]_{\tau=t-H+1}^{t}$, a table of 60 rows by 4 columns.

3.2 What Drives the Drift?

On the motor-off data, **wind speed alone** explains little: a linear fit, drift speed = $0.72\,v + 8.1$ (m/min, v in kn), has $R^2 \le 0.15$ for speed measured over any window from 1 to 60 minutes, best at 10 (shorter adds GPS noise, longer blurs changes). The slope is 2.3% of the wind speed, and the 0.26 kn of drift left with no wind shows that current matters at least as much as wind. **Adding wind direction and the past H minutes of drift** raises R^2 to 0.83 at $H = 60$: the next hour is 0.84–0.92 of the last, turned slightly clockwise. Past drift captures current and tide without measuring them, so every model takes it as input. The fit weakens as H grows ($R^2 = 0.93$, 0.91, 0.83 at $H = 1$, 30, 60), since wind and current have longer to change, but a pilot needs an hour of warning, so we keep $H = 60$. These R^2 are on training data; §4 tests on an unseen deployment.

3.3 Cleaning and Splitting the Data

Gaps of up to 5 missing minutes are filled by linear interpolation. A minute is *valid* if the motor is off, no value is missing, and the vessel moved less than 93 m (about 3 kn; faster means it was being towed, not drifting). A time t becomes a training sample $(\mathbf{X}_t, \mathbf{y}_t)$ only if all 120 minutes from $t - 59$ to $t + 60$ are valid, so neither the input nor the answer includes motor use. Five deployments (Apr–Oct 2026, 85,915 minutes) give 23,319 samples.

Samples one minute apart share 59 of their 60 input minutes, so a random train/test split would put near-copies of test samples in training. We therefore split by whole deployment: **train** (Apr, May–Jun, Aug; 16,471 samples), **validation** (Oct 3–13; 2,289), used only to decide when to stop training, and **test** (Oct 16–23; 4,559), never used for any choice. Inputs and targets are rescaled to mean 0 and standard deviation 1 using training data only.

3.4 Models

Each model is a different way to compute g in Eq. 1. **(1) Persistence** (no training): the next hour repeats the last, $\hat{\mathbf{y}}_t = \sum_\tau \Delta\mathbf{p}_\tau$. Current and tide change slowly, so this is a strong bar to beat. **(2) Linear regression**: $\hat{\mathbf{y}}_t = \mathbf{A}\,\mathbf{z}_t + \mathbf{c}$, the fit of §3.2 plus mean wind: $\mathbf{z}_t$ summarises the window in 6 numbers (past-hour movement, current

Algorithm 1 Drift prediction: build data, train, predict

Input: deployments $\{\mathcal{S}_k\}$, horizon H, model g_Θ
Output: trained g_{Θ^*}; predicted position $\hat{\mathbf{p}}_{t+H}$

for all deployments $\mathcal{S}_k$ **do** ▷ build samples
  fill gaps ≤ 5 min; $\mathbf{p}_\tau \leftarrow$ metres from station
  $\mathbf{w}_\tau \leftarrow -v_\tau(\sin\theta_\tau, \cos\theta_\tau)$ ▷ wind push
  $\text{valid}_\tau \leftarrow$ motor off $\wedge$ no missing $\wedge$ $\|\Delta\mathbf{p}_\tau\| < 93$
  for all t where minutes $t{-}H{+}1..t{+}H$ are all valid **do**
    add $(\mathbf{X}_t,\ \mathbf{y}_t = \mathbf{p}_{t+H} - \mathbf{p}_t)$ to the split of $\mathcal{S}_k$
rescale with train statistics; seed $\leftarrow 0$; best $\leftarrow \infty$
for epoch = 1..80 **do** ▷ train
  for all mini-batches B of train **do**
    $\Theta \leftarrow \text{Adam}\big(\Theta, \nabla_\Theta \frac{1}{|B|}\sum_B \|g_\Theta(\mathbf{X}) - \mathbf{y}\|^2\big)$
  if val loss < best **then** best $\leftarrow$ val loss; $\Theta^* \leftarrow \Theta$
  else if no improvement for 10 epochs **then break**
return $\hat{\mathbf{p}}_{t+H} = \mathbf{p}_t + g_{\Theta^*}(\mathbf{X}_t)$ ▷ predict

wind push, mean wind push over the hour) and $\mathbf{A} \in \mathbb{R}^{2\times 6}$, $\mathbf{c}$ are fitted by least squares. Its weights can be read directly. **(3) LSTM** [2] (2 layers, 64 hidden units, dropout 0.2): reads the 60 rows of $\mathbf{X}_t$ one minute at a time, updating a memory vector $\mathbf{h}$; the final $\mathbf{h}_t$ is mapped to $\hat{\mathbf{y}}_t = \mathbf{W}\mathbf{h}_t + \mathbf{b}$. Unlike the 6-number summary, it can see whether the drift is turning or slowing down. **(4) Transformer** [6] (2 layers, width 64, 4 heads, feed-forward 256, dropout 0.1): each minute becomes a 64-number vector plus a code for its position in time, $\mathbf{e}_\tau = \mathbf{W}_e\mathbf{x}_\tau + \text{PE}(\tau)$. Self-attention layers let every minute compare itself with every other; the outputs are averaged over the hour and mapped to $\hat{\mathbf{y}}_t$. A more complex model is only worth using if it beats the simpler one on unseen data.

3.5 Training, Inference and Scoring

The LSTM and transformer minimise mean squared error with Adam [3] (learning rate 10^{-3}, batch 256, gradient clip 1.0, seed 0). After each pass over the training data (epoch) we keep the weights with the lowest validation loss and stop after 10 epochs without improvement, or after 80. This matters because the 23k overlapping windows hold only about 400 independent hours of drift, and both networks start memorising within a few epochs. These settings are common defaults, not tuned. At inference the output is scaled back to metres and added to $\mathbf{p}_t$.

We score each prediction by its **endpoint error** $e_t = \|\hat{\mathbf{p}}_{t+H} - \mathbf{p}_{t+H}\|_2$, the distance in metres between predicted and true position. We report the median (a typical miss, robust to GPS glitches) and the 90th percentile (a bad-case miss, usable as a safety radius).

Worked example. On Oct 19 at 21:15 (test deployment) the vessel had moved $(774, 555)$ m east/north in the past hour and was slowing down. Persistence predicts $(774, 555)$ m again and the LSTM $(460, 353)$ m; the vessel actually moved $(283, 194)$ m. The endpoint errors are 609 m and 238 m: the LSTM noticed the slowdown.

4 Implementation

The code is in Python (scikit-learn, PyTorch). The fits of §3.2 are in `jake/linear-reg/`; the pipeline is in `rowan/`, one file per stage, matching Algorithm 1: `data_prep.py` (representation, cleaning, split, endpoint error; lines 1–6), `linear_reg.py` (persistence, linear regression), `nn_models.py` (LSTM, transformer, training loop; lines 7–13) and `compare.py` (results table, chart). Outputs go to `outputs/`. The README covers the project, installation, quick start and results; dependencies are pinned in `requirements.txt` and the data are in the repository. The quick-start script `quickstart.sh` creates the environment on first use and runs all five steps in order (about 5 minutes on a laptop CPU):

```
git clone https://github.com/jakecush1/autonomous-pilot-ML
cd autonomous-pilot-ML/usv-intervention/solution/rowan
bash quickstart.sh
```

Reproducibility. Seeds are fixed to 0 and the split is fixed by deployment, so the quick start on CPU reproduces Table 1 exactly (confirmed on three runs).

First results. Table 1 shows the pipeline runs end to end. Persistence already halves the error of assuming no drift. Linear regression's training R^2 of 0.83 (§3.2) falls to 0.77 on the unseen deployment and does *not* beat persistence, while the LSTM and transformer both do, by about 16%, and are tied with each other. Milestone 4 will tune the settings, test more seeds and horizons, and turn the prediction into a time until the vessel reaches the boundary [1], toward an intervention probability [4].

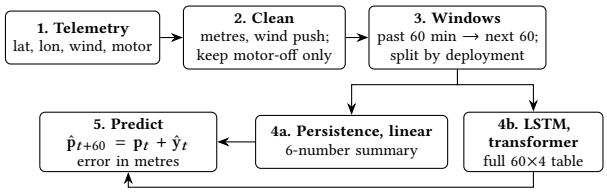


Figure 1: Pipeline: each model maps a 60-minute motor-off window of telemetry to the next hour's movement.

Table 1: Endpoint error after 60 min, test deployment.

Model	Median (m)	90th pct. (m)	R^2
No drift ($\hat{\mathbf{y}} = 0$)	586	1515	0.00
(1) Persistence	294	570	0.80
(2) Linear regression	318	631	0.77
(3) LSTM	247	497	0.85
(4) Transformer	**246**	**477**	**0.86**

References

[1] Spencer August Dugan and Ingrid Bouwer Utne. 2024. Development of a Risk Indicator for Ship Drifting Groundings. In *Proc. PSAM17 & ASRAM2024.* https://www.iapsam.org/PSAM17/program/Papers/PSAM17&ASRAM2024-1377.pdf

[2] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Comput.* 9, 8 (1997), 1735–1780. doi:10.1162/neco.1997.9.8.1735

[3] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *Proc. ICLR 2015.* https://arxiv.org/abs/1412.6980

[4] Susanna Dybwad Kristensen, Yiliu Liu, and Ingrid Bouwer Utne. 2022. Dynamic Risk Analysis of Maritime Autonomous Surface Ships. In *Proc. PSAM 16.* https://www.iapsam.org/PSAM16/papers/SU247-PSAM16.pdf

[5] Yuxuan Song et al. 2024. The Prediction and Dynamic Correction of Drifting Trajectory for Unmanned Maritime Equipment Based on Fully Connected Neural Network (FCNN) Embedding Model. *J. Mar. Sci. Eng.* 12, 12 (2024), 2262. doi:10.3390/jmse12122262

[6] Ashish Vaswani et al. 2017. Attention Is All You Need. In *Proc. NeurIPS 2017.* 5998–6008.